课程笔记：强化学习系统

Bowen-mantou（整理）

CMU 11-768: AI Agents（2026 年秋季）
主讲：CMU 11-768 课程组

2026 年 10 月 23 日

视频作者/频道：常青藤名校课（B 站）
发布日期：2026-09-09
视频时长：66:27
视频链接：https://www.bilibili.com/video/BV1a6Yx62EH4?p=26
PDF 制作 Skill：https://github.com/wdkns/wdkns-skills

目录

1 本讲定位：从算法走到「系统」 2

1.1 第三讲的任务：把算法装进机器 . . . 2
1.2 一个 LLM RL 框架至少要支持哪些操作 . . . 2
1.3 为什么不能「用 PyTorch 手写」了事 . . . 2
1.4 一个训练步的完整生命周期 . . . 3
1.5 本章小结 . . . 3

2 双引擎架构：推理与训练各自的优化 3

2.1 推理侧：KV cache 是显存的主要去向 . . . 3
2.2 推理侧：跨请求的 prefix 复用甚至更重要 . . . 4
2.3 推理侧：continuous batching、chunked prefill 与 PD 分离 . . . 4
2.4 推理侧：投机解码在 RL 里的特殊问题 . . . 4
2.5 训练侧：激活检查点与训练侧的 prefix 复用 . . . 5
2.6 训练侧：五种并行分片策略 . . . 5
2.7 引擎交互之一：权重同步为什么能只传 20–30 GB . . . 5
2.8 引擎交互之二：MoE 的「路由重放」(R3) . . . 6
2.9 本章小结 . . . 6

3 Agentic RL 的「尖锐细节」(The Sharp Bits) 6

3.1 序列扩展：为什么「压缩上下文」的 harness 更难训 . . . 6
3.2 Chat template 的坑：推理链被悄悄删掉 . . . 7
3.3 Token-in, token-out：不要用字符串在引擎之间传数据 . . . 7
3.4 异步 RL 在 agent 训练里为什么更重要 . . . 7
3.5 In-flight 权重更新：生成到一半，权重换了 . . . 8
3.6 复杂 harness：递归多 agent 与「一棵树怎么进 batch」 . . . 8
3.7 本章小结 . . . 8

4 框架的系统设计选择 8

4.1 Harness-agnostic：用一个「中间人」隔离 harness 与训练框架 . . . 8
4.2 SPMD：一个程序跑遍所有卡，简单但不灵活 . . . 9
4.3 Everything-as-a-service：控制器 + 标准协议 . . . 9
4.4 服务化带来的三种新能力 . . . 10
4.5 本章小结 . . . 10

5 总结与延伸 10

5.1 讲者的收尾：一整讲其实在讲三件事 . . . 10
5.2 一张表把系统问题映射到排查手段 . . . 11
5.3 课堂最后一个问答：BF16 更新被丢掉，会不会损失信号? . . . 11
5.4 延伸阅读 . . . 11

1 本讲定位：从算法走到「系统」

1.1 第三讲的任务：把算法装进机器

三讲强化学习部分的最后一讲，主题从「算法」转向「系统」。前两讲讨论的是策略梯度、信用分配、异步训练与稳定性；这一讲要回答的是一个更工程化的问题：**如果让你从零搭一个能训练 LLM agent 的 RL 框架，你需要哪些部件？它们为什么这样切分？**

讲者给本讲的目标定得很具体：不是教你从头实现一个框架，而是**带你走一遍「如果你用 RL 框架训 agent、或者要 fork 修改某个框架时会遇到的东西」**。所以本讲的内容结构是「总览 → agentic RL 特有的坑 → 框架设计选择」，这也正好对应后面三节。

1.2 一个 LLM RL 框架至少要支持哪些操作

讲者先抛出一个问题：**要训练这样一个模型，框架里必须有哪些核心操作？**课堂给出的答案可以归成三类（图 **??**）：

1. **前向（forward）**：既用来算 loss，也用来做 rollout 采样（agent 生成动作、与环境交互都要前向）；
2. **反向（backward）**：用来自动微分、计算梯度；
3. **优化器步（optimizer step）**：把算出来的更新真正写回模型权重。

如果把策略梯度写成一句话，它正好把这三件事串起来：

$$\nabla_\theta J(\theta) = \mathbb{E}\Big[\sum_t \big(R(\tau) - b(h_t)\big)\,\nabla_\theta \log \pi_\theta(a_t \mid h_t)\Big]$$

逐项解释：θ 是策略参数；τ 是轨迹；$R(\tau)$ 是轨迹奖励；$b(h_t)$ 是基线（上一讲讨论过组内均值、价值函数等都是它的实现）；$\nabla_\theta \log \pi_\theta(a_t \mid h_t)$ 是「让这个动作更可能出现的梯度方向」。括号里的 $R(\tau) - b(h_t)$ 是权重，$\nabla_\theta \log \pi_\theta$ 这一项正是要反向传播的东西。

但对 **agentic** RL 来说，这三件事远远不够。讲者补充了三个 agent 特有的部件：

- **环境（environment）**：策略在环境里执行动作、拿回观测，所以框架必须能「step 环境」；
- **奖励函数与优势（reward & advantage）**：需要对采样到的轨迹做验证、打分、算优势；
- **推理的结构（agent harness）**：agent 如何组织 prompt、工具调用、多轮交互，这决定了推理请求长什么样。

1.3 为什么不能「用 PyTorch 手写」了事

很多人第一次接触这些框架时都会问：这些操作映射到 PyTorch 不就是 `model.forward`、`loss.backward`、`optimizer.step` 吗？为什么需要 SkyRL、Miles、veRL、PrimeRL、Slime 这么多框架？

讲者的回答是：**因为推理和训练是两种负载特性完全不同的 workload**。如果只写一个朴素的 PyTorch 循环，你会在两边同时损失性能：

- **推理侧**：自回归地一个 token 一个 token 解码，是 **decode 为主**的负载；不需要 autograd、不需要保存计算图，但需要大量显存给 **KV cache**；

- **训练侧**：整条序列都已知，可以**并行**处理（更像 prefill）；必须保存激活值和计算图才能反向传播。

两种负载的最优实现方式差别太大，于是几乎所有成熟框架都采用**双引擎**架构：一个专门做推理（例如 vLLM、SGLang），一个专门做训练（例如 Megatron、FSDP2），中间通过「权重同步」和「轨迹数据」两个通道连接（图 **??**）。

1.4 一个训练步的完整生命周期

把双引擎拼起来之后，一个训练步长这样（图 **??**）：

1. **sample**：推理引擎与 agent harness 交互，在环境里执行动作、拿到观测，采出一条条轨迹；
2. **verify**：环境或验证器给出奖励信号（测试是否通过、答案是否正确等）；
3. **forward**：训练引擎把整条轨迹并行地重新前向一遍，**保留激活与计算图**；
4. **backward**：反向传播算梯度；
5. **optimizer step + weight sync**：更新参数，并把新权重同步回推理引擎，开始下一轮。

讲者特别提醒：推理引擎和训练引擎是**同一个模型的两套实现**，它们的 kernel 或前向计算方式可能略有不同，因此即使权重完全相同，采样出来的分布也会有细微差异——也就是说，框架在设计上**天然带一点 off-policy**。所以上一讲讨论的重要性采样、裁剪等 off-policy 校正在这里不是可选项，而是必需品。

1.5 本章小结

- LLM RL 框架的核心操作是前向、反向、优化器步；agentic RL 还要加上环境交互、奖励验证、harness 结构。
- 不能用单一 PyTorch 循环替代框架：推理与训练的负载特性（decode vs prefill、是否保存计算图、显存去向）根本不同，所以采用推理/训练双引擎。
- 一个训练步是「采样 → 验证 → 前向 → 反向 → 更新与权重同步」的循环；两套引擎的计算差异天然带来轻微 off-policy，需要显式校正。

2 双引擎架构：推理与训练各自的优化

2.1 推理侧：KV cache 是显存的主要去向

自回归解码时，每生成一个新 token 都要对全部历史做注意力；如果每步都重算 prefix 的 K/V，计算量会随长度平方增长。**KV caching** 就是把这些算过的 key/value 存进显存，用「显存」换「计算」。

讲者给了一个数量级估算，用来说明推理侧显存压力有多大：**Llama-3-8B、BF16 精度下，64 条 32K token 的轨迹，KV cache 大约要 256 GB**（讲者口头估算，具体数值随实现而变，细节待核对）。而且这还只是一个 8B 的小模型；在 RL 场景里，batch size 可以非常大，KV 数据会更多。

这也解释了为什么双引擎架构有优势：如果训练和推理共用一个引擎，你既想把激活值留在显存里做反向，又想给 KV cache 留足空间，显存会严重冲突；分开之后，推理引擎可以几乎把显存都花在 KV cache 上。

2.2 推理侧：跨请求的 prefix 复用甚至更重要

KV cache 解决的是「一个请求内部」的重复计算；agentic RL 里还有大量「**请求之间**」的重复：一次 agent 轨迹的每一步都是一个独立的 LLM 请求，它们共享同一段系统 prompt、同一个任务 prompt，甚至前一步的完整历史就是后一步的 prefix。

一个关键事实

「轨迹的每一步是一个独立的 LLM 请求，不是一次连续的推理。」——所以在 agentic RL 里，如果引擎不支持 **prefix caching**（前缀缓存），同一段 prefix 会被反复 prefill 很多次，浪费是成倍的。

主流的实现方式有两种：SGLang 的 **RadixAttention**（用基数树管理前缀、复用粒度细）和 vLLM 的 **automatic prefix caching**（按 block 内容哈希复用）。讲者把「跨请求复用」列为推理引擎对 agentic RL 最重要的能力之一。

2.3 推理侧：continuous batching、chunked prefill 与 PD 分离

continuous batching（连续批处理）解决的是「长度不齐带来的空转」（图 **??**）：朴素批处理要等整批里最长的请求做完，短请求完成后槽位就空转；连续批处理则在**每个 decode step 之后重新做调度决策**——刚完成的请求立刻退出 batch，腾出的容量马上给等待中的新请求。

但实际的推理负载不只有 decode，还有 **prefill**（把 prompt 一次性并行处理）。长 prompt 的 prefill 会阻塞很多 decode 请求，所以现代引擎会做两件事：

- **chunked prefill**：把长 prompt 切成块，与 decode 交错处理，避免一次性长时间占住 GPU；
- **disaggregated prefill/decode（PD 分离）**：把一部分 GPU 专门用于 prefill、另一部分专门用于 decode。讲者解释它的另一个好处是**异构硬件适配**：例如 NVIDIA 服务器里既有适合高吞吐 prefill 的硬件（讲者口述提到 Groq LPU、Blackwell/Rubin 一类，名称听记）、也有适合 decode 的 GPU，分离后可以为不同阶段挑选更合适的硬件；代价是 prefill 与 decode 之间要传递 KV（通常通过队列/服务完成）。

2.4 推理侧：投机解码在 RL 里的特殊问题

speculative decoding（投机解码，图 **??**）用一个更小更快的 draft 模型先连续猜几个 token，再让大模型并行验证；配合拒绝采样，可以在**保持与原大模型分布等价**的前提下加速。

讲者指出它在 RL 训练中有一个特殊陷阱：训练初期 acceptance rate 可能很高，但 RL 在不断改模型，**基座模型一漂移，draft 模型的接受率就会掉下来**。常见的应对是用辅助目标（SFT/蒸馏）在训练过程中**持续更新 draft 模型**，让它跟上基座。

2.5 训练侧：激活检查点与训练侧的 prefix 复用

训练侧的核心矛盾是**激活值显存**。反向传播需要前向时算过的激活，全存下来显存会爆；**activation checkpointing**（激活检查点，图 **??**）的做法是只保留每层输入、丢弃中间激活，反向时再**选择性重算**缺失的激活。

讲者给了量化代价：这相当于多做一次前向，**大约多花 33% 的时间**，但省下大量显存，是「显存受限时的好朋友」。实践中还分 **full** 与 **selective**（只对部分层做检查点）两种模式。

推理侧的 prefix 复用在训练侧同样存在，而且更复杂：训练要保留激活并做反向。讲者提到的代表方案是 Snowflake 的 **Arctic RL** 里的 **Zorro**，以及其中使用的 **dynamic tree attention**：把一个 batch 的公共前缀组织成**前缀树**，沿树做 DFS——先对公共祖先做前向，进入某一分支做前向与反向，反向完成后立刻**弹出计算图、释放激活**，再回退走另一分支。这样显存中同时保留的激活**永远不超过一条根到叶的路径**，既复用了 prefix，又没有把显存撑爆。

2.6 训练侧：五种并行分片策略

模型大到单卡放不下时，训练引擎要把模型/数据切到多卡上。讲者按「切什么」把主流方法过了一遍，这里汇总成表 1。

策略	切分对象	通信特点	适用场景
DP（数据并行）	全局 batch 切成 micro-batch，每卡一份完整模型	每步梯度 all-reduce，一次、可跨机	默认首选；有「按 token 预算装箱」的 micro-batch 划分算法
PP（流水线并行）	按层切成 stage，不同 rank 放不同层	相邻 stage 传激活；用 micro-batch 填充流水	层数多、单卡装不下整模型
EP（专家并行）	MoE 的不同专家放到不同 rank	路由带来的 all-to-all	MoE 模型
TP（张量并行）	把单层计算(矩阵乘/注意力头）切开	通信量大，通常只在节点内 NVLink 上	单层都放不下时；应尽量少用
CP（上下文并行）	按序列维切，各 rank 持一段 token	注意力需要交换 KV（环形传递）	超长上下文

表 1: 五种并行分片策略对照。讲者的经验法则是：**尽量把并行度花在 DP 上，减少 TP 之类通信密集的方案**。

讲者给的经验法则值得单独记住：**优先最大化数据并行，减少张量并行的比例**——因为 TP 的通信开销与层内依赖最强，通常限制在单机内；而 DP 可以跨机扩展。分片策略对推理同样重要，只是推理侧的约束从「激活」变成了「KV cache」。

2.7 引擎交互之一：权重同步为什么能只传 20–30 GB

两个引擎之间的第一个通道是**权重同步**。直觉上这件事很贵：一个 1T 参数模型，BF16 下约 **2 TB**；如果每次更新都把全量权重广播到所有推理 rank，根本不可行（讲者也提到这正是「跨集群训练 + 存到 S3 再下载」这类做法的现实动机）。

但讲者指出一个关键观察：**RL 的更新非常稀疏**。在 BF16 精度下，两次更新之间**约 99% 的参数是逐比特相同的**——因为每条轨迹的监督信号很少、更新幅度极小，很多参数在 BF16 下根本没有变化。于是可以**只同步 delta**：需要传输的数据从 2 TB 降到大约 **20–30 GB**，就完全可行了。

为什么「99% 参数没变」是合理的

BF16 的尾数只有 7 位（讲者在 Q&A 里确认），表示精度有限；RL 里每条轨迹的信号又很稀疏，很多参数的更新量小于 BF16 能表示的最小变化，于是它们在数值上保持不变。讲者对这个现象的实用判断是：**如果某次更新小到被 BF16 丢掉、但它重要，下一次训练步会出现更大的更新补回来**；而用 32 位训练在显存上又不可行——所以 BF16 训练 + delta 同步是现实的最优解。

同步完权重还有一层工程细节：训练引擎的分片布局与推理引擎的分片布局通常不同，所以需要**重新分片（re-shard）**，让权重符合推理引擎的切分方式。

2.8 引擎交互之二：MoE 的「路由重放」（R3）

第二个不显眼但很要命的交互是 **MoE 路由**。MoE 前向时要为每个 token 选择专家；由于实现细节差异、甚至只是路由打分出现平局，**训练引擎选中的专家可能与推理引擎当时选中的不同**——这就让训练的前向与推理的前向不一致，产生额外的 off-policy。

讲者给的解法非常直接（图 **??**）：在 rollout 阶段**记录每个 token 的路由决策**，随轨迹数据一起传给训练引擎；训练引擎前向时**丢弃自己的路由结果、强制重放这些记录**。这个技术被称为 **Rollout Routing Replay（R3）**，讲者说它对 MoE 训练稳定性帮助很大，并给了一句很实用的排查建议：**如果你在训 MoE 模型时发现 loss 或 reward 不涨，先检查有没有开 R3**。

2.9 本章小结

- 推理侧优化围绕 KV cache 展开：KV caching、prefix caching（RadixAttention / automatic prefix caching）、continuous batching、chunked prefill、PD 分离、投机解码（RL 下要在线更新 draft）。
- 训练侧优化围绕激活显存展开：activation checkpointing（约 33% 时间换显存）、训练侧 prefix 复用（Arctic RL/Zorro 的 dynamic tree attention）、五种并行分片。
- 引擎交互有两件事：权重同步（利用更新稀疏性，2 TB → 20–30 GB 的 delta）与 MoE 路由重放（R3，MoE 训练稳定性必备）。

3 Agentic RL 的「尖锐细节」（The Sharp Bits）

3.1 序列扩展：为什么「压缩上下文」的 harness 更难训

agent harness 有两种典型设计（图 **??**）：

- **harness A（追加式）**：每一步都把新的动作/观测**追加**到历史后面，历史前缀永远保持不变；
- **harness B（压缩式）**：每做几步就把之前的动作与观测**总结压缩**成一段 summary，再从 summary 继续。生产系统里很常见（讲者点名 Claude Code、OpenHands 都做 context compaction）。

课堂投票里多数人认为 B 更好（因为上下文更短），但讲者的答案是：**从训练效率看 A 更好**——只有在序列实在太长、必须压缩时才用 B。

原因在 batch 的组织方式（图 **??**）：harness A 的前缀完全连续，**整条轨迹可以当作一个数据点**进 batch，一次 prefill 就够；harness B 一旦压缩，就把轨迹断成了两段独立序列，**必须拆成两个**

数据点——于是任务 prompt 被重复 prefill 两次，计算浪费。要缓解这一点，就得依靠前面讲过的 training 侧 prefix 复用技巧（如 dynamic tree attention 把多个数据点组织成树来复用计算）。

3.2 Chat template 的坑：推理链被悄悄删掉

比上下文压缩更隐蔽的问题是 **chat template**。讲者以 Qwen3 / Qwen3.5 的默认 chat template 为例（图 **??**）：开启 reasoning 后，模型会先输出一段思维链再给动作；但当新的 user turn 或 harness 输出进来时，**默认模板会把历史里的 reasoning token 删掉**。

这在工程上可以理解（节省上下文窗口、避免干扰），但在 RL 训练里它等价于「每一步都做一次上下文压缩」，**直接破坏了序列扩展性质**：prefix 不连续、历史被改写。于是他给出了一个很实用的结论：**很多开源 RL 框架会自带定制的 chat template（现在常被称为 renderer）**，用来保证前缀不丢；一些训练服务还提供开关（例如「跨轮保留 reasoning」「完全不要 reasoning」）。讲者还提到课程作业里训练小 RL 框架时会提供保留前缀的 Qwen3.5 模板（课程内部资料）。

3.3 Token-in, token-out：不要用字符串在引擎之间传数据

早期框架在推理引擎与训练引擎之间传轨迹时，很自然地用**字符串**（因为 OpenAI 风格的 chat completions API 就是字符串消息，prompt 工程也都在字符串上做）。讲者说，**几乎所有框架后来都发现这是错的**，正确的做法是 **token-in, token-out**：保存并传递**精确的 token 序列**（图 **??**）。

原因有两层：

- **字符串侧的微小变换会放大成 tokenization 差异**：训练引擎的 chat template 只要插入一个空格或换行，重新 tokenize 出来的 token 序列就可能与推理时不同，于是训练和采样用的根本不是同一串 token——直接 off-policy。
- **tokenization 不是一一对应的**：文本到 token 是确定的，但 token 到文本再回 token 不唯一。讲者举了一个例子：某个词在推理时被模型输出成「1 个 token + 5 个 token」的组合，但重新 tokenize 同一段字符串可能得到「4 号 token」——同一个字符串对应不同 token 序列，训练目标就被悄悄改掉了。

讲者说这类问题在早期框架里造成过很多莫名其妙的 **loss 尖刺与训练不稳定**，现在几乎所有 RL 框架都会保存精确 token 并直接传给训练引擎。

3.4 异步 RL 在 agent 训练里为什么更重要

上一讲从「稳定性」角度讲了异步 RL；这一讲从「系统」角度再讲一遍（图 **??**）：agent 轨迹长度差异极大，同步训练时 GPU 大量时间在等最长的 rollout（straggler）；异步训练让推理与权重更新**重叠**，更长的轨迹可以拖到下一批处理，短轨迹则可以先被用上更新。

讲者还给了两个具体的系统设计事实：

- **同步训练的常见做法是 co-locate**：8 张卡先全部用来训练，再让训练引擎「睡眠」、把同一批卡切给推理引擎，交替使用；
- **异步训练可以拆分 GPU 配比**：因为推理与训练同时活跃，可以给更吃紧的一侧更多卡。讲者说他在实践中常用 **3:1** 的比例（例如 6 个节点做推理、2 个节点做训练），因为长程 agent 任务里 rollout 往往是瓶颈。

3.5 In-flight 权重更新：生成到一半，权重换了

异步 RL 有一个很「反直觉」的操作：推理引擎正在为某些请求生成时，训练完成了新一轮更新，需要**中途换权重**。讲者把这类操作称为 **in-flight weight updates**，并指出推理引擎为此专门演进出了接口。

以 SGLang 的文档为例（图 **??**），引擎提供 **pause generation** 接口，其中 **retract** 模式很有用：它不只是暂停生成，还会**清空 KV cache**——因为参数更新后，prefix 的 KV 已经**过期**，必须用新模型重算。讲者总结：vLLM 与 SGLang 都在为 RL 框架的需求持续演进。

3.6 复杂 harness：递归多 agent 与「一棵树怎么进 batch」

agentic RL 的最后一个系统难点来自 harness 本身越来越复杂。讲者展示了他近期论文里训练**递归多 agent 系统**的推理模式：一个 root agent 把任务委派给子 agent，子 agent 还可以继续委派——这来自「递归语言模型」的思路。再叠加前面说的上下文压缩，推理结构就变成一棵树。

于是框架必须回答一个没有标准答案的问题：**整棵树的一次 rollout 算一个数据点，还是把每个子 agent 的轨迹当成独立数据点进 batch？**不同的选择会影响 prefix 复用、loss mask、优势广播与 batch 组织。讲者没有给唯一答案，只是把它作为「agent harness 复杂化带来的系统设计问题」摆出来。

3.7 本章小结

- 序列扩展：追加式 harness（A）对训练更友好，压缩式（B）会把一条轨迹拆成多个数据点、产生重复 prefill；chat template 若删除历史 reasoning，等价于偷偷做压缩。
- 引擎之间必须 **token-in, token-out**：字符串重 tokenize 会导致 off-policy 与 loss 尖刺。
- 异步 RL 是 agent 训练的系统默认选择：overlap、拆分 GPU 配比（常见 3:1）、in-flight 权重更新（pause/retract 并清 KV cache）。
- 递归多 agent 与 context compaction 让「如何组织 batch」成为新的开放设计问题。

4 框架的系统设计选择

4.1 Harness-agnostic：用一个「中间人」隔离 harness 与训练框架

agent harness 生态极其多样（讲者点名 Codex、Claude Code、OpenHands、PydanticAI（Py）、Hermes、Prime Agent 等，且每天都在增加）。如果每加一个 harness 就要改 RL 框架，维护成本会爆炸。

讲者介绍的流行做法是 **harness-agnostic**：在 harness 与推理引擎之间放一个**中间人/代理服务**，它**模拟 harness 已经在用的推理 API**（chat completions 或 responses API），把请求转发给推理引擎、把响应转发回 harness；同时**在中间把训练需要的一切记录下来**（logprob、R3 的路由数据等）。这样 harness 侧零改动，框架只维护一个代理；这个代理甚至可以打包成一个「推理服务」对外暴露 endpoint。

4.2 SPMD：一个程序跑遍所有卡，简单但不灵活

讲者接着讨论分布式代码的组织方式。最直观的是 **SPMD**（Single Program Multiple Data）：所有 GPU rank 跑**同一份 Python 程序**，靠 `rank` 做条件分支，让不同 rank 执行不同角色（伪代码见下方）。

代码长这样（示意，非原文）：

```
train = dist.new_group([0, 1, 2, 3])
infer = dist.new_group([4, 5, 6, 7])

if rank < 4:
    batch = next_batch_shared(train)
    forward_backward(train)
    optimizer_step()
else:
    while True:
        refresh_weights_if_ready(infer)
        trace = rollout(prompt, infer)
        enqueue_trajectory(trace, infer)
```

Listing 1: SPMD 风格的伪代码（据讲者幻灯片整理）：所有 rank 跑同一程序，按 rank 分组执行训练或推理分支。

讲者对这个模式的评价很中肯：优点是**直观、全局可见、改起来直接**（完整逻辑在一个程序里）；缺点是在大规模、多角色、异构硬件的场景里**非常不灵活**。他举的实例是 AReaL：它在 1.0 之前采用 SPMD，之后转向了下面要讲的服务化架构。

4.3 Everything-as-a-service：控制器 + 标准协议

另一种更现代的模式是「**一切皆服务（everything as a service）+ 单控制器**」：训练服务、推理引擎、权重同步服务、agent harness、环境服务各自作为独立服务存在，一个**轻量的 CPU 控制器**负责编排（图 **??**）。

讲者列出的优点很具体：

- **分离关注点**：每个组件都极其复杂（推理、训练各有自己的优化体系），拆开后可以独立构建、维护、升级；
- **标准协议**：harness 与工具/环境之间可以用 **MCP**，推理侧用 chat completions / responses API；
- **轻量控制器**：编排循环可以是纯 CPU 的，不需要 GPU、不引入复杂依赖；
- **模块化可替换**：只要协议一致，推理引擎可以换 vLLM/SGLang，训练框架可以换，甚至 harness 也可以换；
- **弹性与异构**：可以同时挂多个推理服务（甚至 AMD 与 NVIDIA 混用），需要时动态加机器。

讲者举了两个例子。一是 **AstroFlow**（Haizhong Zheng 等，2026）——它把 rollout 做成服务：推理服务 + agent + 环境都可以在不同 GPU 上、按需加入；权重同步服务把权重推给所有推理服务。二是 **Tinker API**：一个标准的训练 API，最初由 Thinking Machines 为其托管训练服务提出，现在 Miles、SkyRL 等开源框架也实现它——**同一份训练代码可以在托管服务与自建集群之间迁移**。

4.4 服务化带来的三种新能力

讲者进一步说明，服务化不只是「解耦好看」，它打开了三类以前很难做的事：

1. **多策略训练（multi-policy）**：不同 rollout/agent 可以跑不同权重甚至不同架构的模型，因为可以为不同模型分配不同训练器与推理引擎。这对多 agent 异构训练（不同 agent 用不同模型）很关键。
2. **在线 RL（训练后部署）**：把线上部署的 harness 也当成数据来源，定期把真实使用数据送进训练服务、更新模型再上线。讲者举的例子是 Cursor：其 tab 模型与 Composer 模型都有持续收集用户数据、更新并部署新版本的流程；据其博客，使用 auto 功能时大约**每 5 小时**就可能用到新的 checkpoint（讲者口述引用博客，细节待核对）。
3. **多租户 / 多 LoRA 训练**：托管训练服务面对很多用户、每人一套 LoRA。Tinker 最早做出这种模式：一个训练 client 对应一次 run，一个采样 client 对应某个 LoRA 权重检查点；底层用 **Grouped-GEMM / SGMV** 等专用 kernel，把多个 LoRA 的推理与反向**并发**执行。

讲者给了这组对比的结论：8 个并发 LoRA run 相比 8 个串行 run，**整体吞吐明显更高**——单个 run 因为共享算力会更慢，但设备利用率上去了。他还提到开源实现（SkyRL 及 Trajectory Labs 一侧的实现）可以自建同样的能力，而 Tinker 是把这套「多租户 + 解耦训练/采样 API」做出名的服务。

4.5 本章小结

- harness-agnostic：在 harness 与推理引擎之间加一个模拟标准推理 API 的代理，顺带采集训练数据；harness 零改动。
- SPMD 简单直接但不够灵活（AReaL 1.0 前用过）；服务化 + 轻量控制器带来分离关注点、标准协议、可替换、弹性与异构硬件、多策略、在线 RL、多租户多 LoRA 等能力。
- 落地建议：先对齐标准 API（Tinker API、chat completions），再按「推理服务 / 训练服务 / 权重同步 / 环境 / harness」拆服务；控制器保持轻量。

5 总结与延伸

5.1 讲者的收尾：一整讲其实在讲三件事

讲者在结尾没有给「标准答案框架」，而是把整讲收敛成三件事：

1. **两种负载必须分开优化**：推理（decode-heavy、KV cache）与训练（prefill-style、激活与反向）各有自己的优化体系；双引擎是主流答案。
2. **引擎交互是新的故障源**：权重同步（利用稀疏性传 delta）、MoE 路由重放（R3）、token-in/token-out、off-policy 校正——这些细节做错，训练就会出现莫名其妙的尖刺与不涨点。
3. **框架设计在向「服务化 + 标准协议」迁移**：harness-agnostic、轻量控制器、可替换组件、多租户多 LoRA、在线 RL，都是这个方向上的能力。

5.2 一张表把系统问题映射到排查手段

症状	可能的系统原因	排查/修复
训练 loss 或 reward 出现尖刺、不稳定	引擎间传字符串导致重 tokenize 不一致；MoE 路由不一致	改为 token-in/token-out；检查 R3 是否开启
推理/训练整体吞吐低、GPU 空转	同步训练等 straggler；batch 未连续调度；KV cache 分配不合理	上异步 RL；continuous batching；prefix caching
权重同步太慢	每次广播全量权重	只传 delta（BF16 下约 99% 参数不变），或走对象存储
训练显存爆	激活未检查点；prefix 重复计算	activation checkpointing（约 33% 时间代价）；training 侧 prefix 复用（dynamic tree attention）
上下文越长越慢、训练效率下降	harness 压缩历史 / chat template 删 reasoning / 轨迹被拆成多个数据点	改用追加式 harness；定制保留前缀的 renderer；序列级 batch 组织
框架难扩展、加一个 harness 要改代码	SPMD 单程序耦合	服务化 + 标准协议；harness-agnostic 代理

表 2: 本讲系统问题的症状—原因—修复对照表。

5.3 课堂最后一个问答：BF16 更新被丢掉，会不会损失信号？

有人问：如果 BF16 精度下很多参数更新小到「没有变化」，是不是在丢训练信号？讲者的回答包含三点：

- 每条轨迹的监督信号本来就很少（outcome reward 更极端），所以单次更新幅度极小，很多参数在 BF16 下确实不变；
- 如果这个更新真的重要，**下一次训练步会出现更大的更新**把它补回来；
- 用 32 位训练在显存上不可行，所以「BF16 训练 + delta 同步」是现实的取舍，而不是疏忽。

本讲最值得带走的一条工程原则

agentic RL 的系统问题，绝大多数不是「某个组件不够快」，而是**两个组件之间的语义没有对齐**：权重怎么同步、路由怎么重放、token 怎么传、prefix 怎么复用、更新到什么程度算「同一个策略」。把接口和语义对齐，性能问题往往就变成可诊断的问题。

5.4 延伸阅读

- 推理引擎：vLLM 的 PagedAttention 与 automatic prefix caching；SGLang 的 RadixAttention 与 pause generation 接口（本讲 in-flight 权重更新的例子来源）。
- 训练系统：Megatron-LM 的 TP/PP/EP 并行；FSDP2 的 ZeRO 式分片；Arctic RL（Snowflake）与 Zorro 的 dynamic tree attention。
- 框架与 API：Tinker（Thinking Machines）的多租户多 LoRA 训练 API；AReaL 的 SPMD → 服务化演进；AstroFlow（Haizhong Zheng 等，2026）的服务化 rollout 设计。